Práctica de aprendizaje automático (parte 3)

Autores:

- Paula Pérez
- Antón Salvadores
- Alejandro Martínez

Profesor: Alberto Suárez Máster en Banca y Finanzas Cuantitativas

Entrega este cuaderno habiendo **ejecutado todas las celdas**. Incluye en la entrega **todos los ficheros necesarios para su ejecución**.

Asegúrate de que la presentación está bien estructurada: Se valorará la **claridad, concisión, y completitud** del informe.

Se trata de una práctica abierta: **sé intrépido y explora**.

1. Describe las características de los datos:
   - Tipo de datos (e.g. https://archive.ics.uci.edu/ml/datasets/statlog+(german+credit+data)))
   - Número de ejemplos de cada una de las clases.
   - Número y tipo (nominales no ordenados, nominales ordenados, numéricos) de atributos.

Esta información se debería utilizar en el análisis; por ejemplo, si se selecciona un atributo para la predicción, ¿es clara su relevancia por el tipo de información que proporciona de acuerdo con nuestro conocimiento experto sobre el área?

2. Detalla la metodología utilizada:
   - Partición de los datos: tamaño de los conjuntos de entrenamiento y test, uso de estratificación en el muestreo.
   - Preprocesamiento: condificación de los atributos, construcción y selección de características, normalización, etc. (¡solo se debe utilizar la información del consjunto de entrenamiento!)
   - Determinación de los hiperparámetros; por ejemplo, mediante búsqueda en rejilla y validación cruzada.
   - Estimación del error de generalización y su incertidumbre.
3. Resume los resultados en gráficas y tablas.
4. Elabora esta información e ilustra tus observaciones con los resultados obtenidos.
5. Proprociona una recomendación final:
   - Tipo de clasificador.
   - Configuración del clasificador (arquitectura, hiperparámetros, etc.) y método de entrenamiento (función de coste, método de optimización, uso de técnicas de regularización,...)
   - para el predictor final, proporciona los errores de entrenamiento, validación cruzada y test.
6. Resume las conclusiones del análisis.

3. Procesamiento de texto y datos no balanceados (3 puntos)

In [1]:

```python
#%load_ext autoreload
#%autoreload 2

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    balanced_accuracy_score,
    confusion_matrix,
    classification_report,
    f1_score
)

import importlib
import tools_assignment_3
importlib.reload(tools_assignment_3)
from tools_assignment_3 import (
    plot_class_distribution,
    report_prediction_distribution,
    compare_models_performance,
    metrics,
    ranking_final
)

sns.set_theme(style="whitegrid", palette="muted", context="notebook")

%matplotlib inline
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['axes.titlesize'] = 16
plt.rcParams['axes.labelsize'] = 14

df = pd.read_csv("train.tsv", sep='\t', index_col="PhraseId")
pd.set_option('display.max_colwidth', None)
df.head(2)
```

Out[1]:

PhraseId	SentenceId	Phrase	Sentiment
1	1	A series of escapades demonstrating the adage that what is good for the goose is also good for the gander , some of which occasionally amuses but none of which amounts to much of a story .	1
2	1	A series of escapades demonstrating the adage that what is good for the goose	2

Tipo de Datos

Se trata de un problema de clasificación de sentimiento en texto (Natural Language Processing - NLP).

- **Dataset:** Sentiment Analysis on Movie Reviews (Kaggle)
- El objetivo es clasificar frases en una escala ordinal de sentimiento.

Diccionario de Atributos:

- **PhraseId:** Identificador único (índice).
- **SentenceId:** Identificador de la oración original (permite agrupar frases del mismo contexto).
- **Phrase:** Texto a clasificar (variable predictora X, tipo: texto no estructurado).
- **Sentiment:** Etiqueta de clase (variable objetivo y, tipo: ordinal).
  - `0` : Negativo
  - `1` : Algo negativo
  - `2` : Neutral
  - `3` : Algo positivo
  - `4` : Positivo
- Nota: Las clases están desbalanceadas, con la clase 'neutral' (2) siendo mayoritaria.

Descripción del dataset

El dataset se compone de reseñas de películas y se utiliza para analizar los sentimientos. En concreto, a través de las reviews de las películas, se clasifica el sentimiento que estas provocan al espectador en cinco clases. Las etiquetas de clases siguen un orden ordinal de 0 a 4, donde la escala de sentimientos se mueve desde el muy negativo hasta el muy positivo.

El dataset se organiza en 4 columnas:

- **PhraseId:** simplemente asigna el número de frases evaluadas, cumple un poco la función del DNI para buscar las frases o subfrases. Son metadatos.
- **SentenceId:** es un identificador de reviews que permite rastrear que frases pertenecen a una sola reseña. El motivo se debe a que el conjunto de datos no solo evalua frases completas (reseñas), sino que cada frase se descompone en fragmentos de distinta longitud. Se codifica mediante números enteros (int). Son metadatos.
- **Phrase (X):** contiene la reseña tanto la frase completa como los fragmento a evaluar, en formato texto (object).
- **Sentiment (y):** es el sentimiento o la etiqueta de clase que se le otorga a la película en función a la reseña. Se codifica como un número entero (int).

En total el dataset está formado por 156.060 observaciones, 8.544 reseñas completas y 4 columnas, pero la dimensión que devuelve `df.columns.tolist()` se reduce a 3 columnas. No obstante, a la hora de evaluar los distintos modelos utilizamos solo la columna de Phrase que hace referencia a nuestro vector de atributos (x) y la columna Sentiment que es el vector de etiquetas (y).

In [2]:

```python
train, test = train_test_split(df[["Phrase", "Sentiment"]],
                               random_state=0)
X_train = train.Phrase
X_test = test.Phrase
y_train = train.Sentiment
y_test = test.Sentiment

print(f"Dimensiones Train: {X_train.shape}, Test: {X_test.shape}")
```

```
Dimensiones Train: (117045,), Test: (39015,)
```

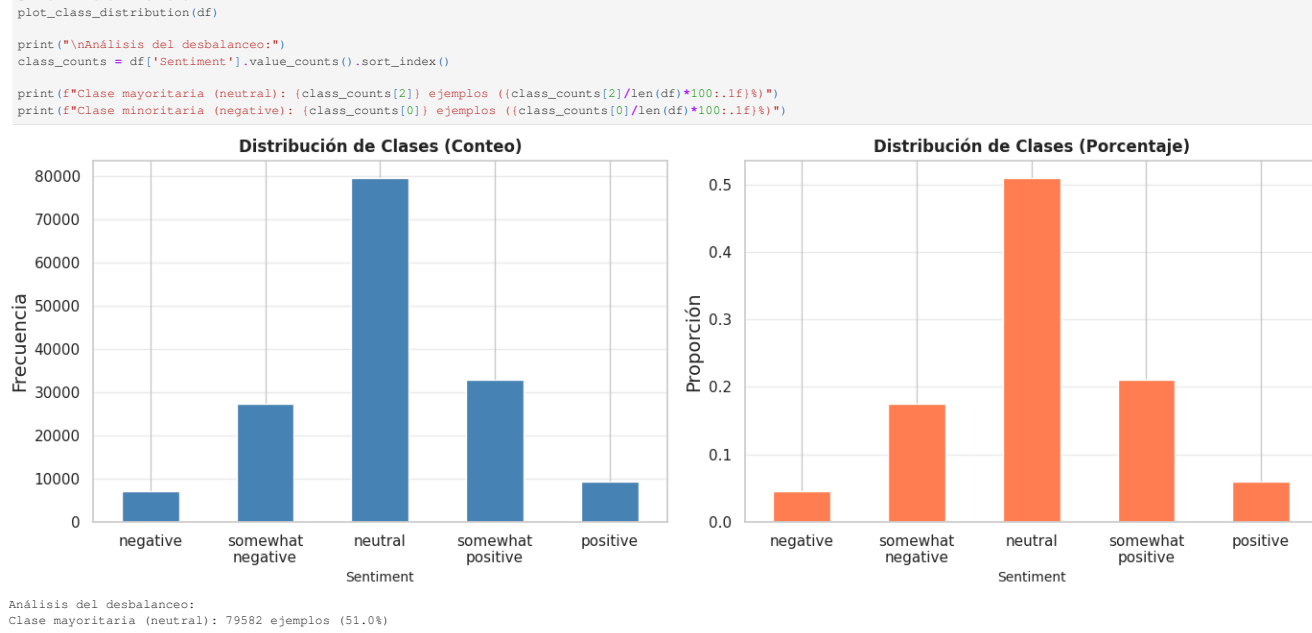
In [3]:

```python
# Análisis de los datos

print(f"Total de ejemplos: {len(df)}")
print(f"Columnas: {df.columns.tolist()}")

print("\nDistribución de clases (Sentiment):")
print(df['Sentiment'].value_counts().sort_index())

print("\nDistribución porcentual (%):")
print(df['Sentiment'].value_counts(normalize=True).sort_index() * 100)
```

```
Total de ejemplos: 156060
Columnas: ['SentenceId', 'Phrase', 'Sentiment']

Distribución de clases (Sentiment):
Sentiment
0     7072
1    27273
2    79582
3    32927
4     9206
Name: count, dtype: int64

Distribución porcentual (%):
Sentiment
0     4.531590
1    17.475971
2    50.994489
3    21.098936
4     5.899013
Name: proportion, dtype: float64
```

In [4]:

```python
# DESBALANCEO DE CLASES
plot_class_distribution(df)

print("\nAnálisis del desbalanceo:")
class_counts = df['Sentiment'].value_counts().sort_index()

print(f"Clase mayoritaria (neutral): {class_counts[2]} ejemplos ({class_counts[2]/len(df)*100:.1f}%)")
print(f"Clase minoritaria (negative): {class_counts[0]} ejemplos ({class_counts[0]/len(df)*100:.1f}%)")
```



```
Análisis del desbalanceo:
Clase mayoritaria (neutral): 79582 ejemplos (51.0%)
Clase minoritaria (negative): 7072 ejemplos (4.5%)
```

Los gráficos de barras evidencian una descompensación visible en la distribución de las etiquetas de clases. Se observa como los valores extremos, 0 y 4, son los que presentan menor frecuencia. Mientras que el sentimiento "neutral" (2) es la clase que más se repite entre las reseñas o fragmentos de reseñas estudiadas, seguido de la clase de "algo positivo" (3) y "algo negativo" (1).

Por tanto, la distribución sugiere la existencia de un sentimiento de indiferencia de los espectadores ante las películas, que ni gustan ni dejan de gustar. Como consecuencia, los resultados de los modelos de clasificación pueden estar sesgados dado la dominancia de la clase 2 frente al resto de clases. Por ello, a la hora de dividir la muestra en el bloque de entrenamiento y test es estadísticamente obligatorio usar `stratify`. Si no lo hiciéramos, correríamos el riesgo de que en el conjunto de Test no cayeran ejemplos de la clase minoritaria (Negative), invalidando nuestra evaluación.

Además, dado el desbalanceo de clases, la métrica de `accuracy` es inadecuada para tratar estos datos. En su lugar usaremos `F-SCORE` y `class_weight='balanced'` para abordar esta desigualdad.

In [5]:

```python
# Stratify para asegurar la proporción de clases

train, test = train_test_split(df[["Phrase", "Sentiment"]],
                               random_state=0,
                               stratify=df["Sentiment"])
X_train = train.Phrase
X_test = test.Phrase
y_train = train.Sentiment
y_test = test.Sentiment
```

Modelos básicos

Vamos a comenzar con una concatenación (*pipe*) de modelos muy básicos y analizaremos si tienen algún problema.

In [6]:

```python
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.pipeline import make_pipeline
from sklearn.tree import DecisionTreeClassifier

count_vectorizer = CountVectorizer(max_features=1000)
decision_tree = DecisionTreeClassifier(criterion='entropy', max_depth=2)
pipe = make_pipeline(count_vectorizer,decision_tree)
```

In [7]:

```python
from sklearn.metrics import accuracy_score

def get_accuracy(pipe):
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)
    return round(accuracy_score(y_pred, y_test),3)
```

In [8]:

```python
get_accuracy(pipe)
```

Out[8]:

```
0.513
```

Veamos qué clases se están prediciendo de verdad

In [9]:

```python
# Etiquetas de entrenamiento
round(pd.Series(y_train).value_counts(normalize=True),2)
```

Out[9]:

Sentiment	proportion
2	0.51
3	0.21
1	0.17
4	0.06
0	0.05

dtype: float64

In [10]:

```python
# Clases en predicción
y_pred = pipe.predict(X_test)
round(pd.Series(y_pred).value_counts(normalize=True),2)
```

Out[10]:

	proportion
2	0.97
3	0.03

dtype: float64

El resultado del `get_accuracy` indica que tan solo el $51,3\%$ de las observaciones del conjunto test clasifican correctamente. Esto es casi más de la mitad de las observaciones que realiza el modelo son erróneas. Es un número muy bajo, ya que, el objetivo es lograr una clasificación perfecta. Uno de los principales motivos puede deberse a la poca profundidad del clasificador `decision_tree` que con tan solo `max_depth = 2` el modelo está underfitting. Esto es, no es capaz de modelizar relaciones más complejas ni obtener información relevante suficiente para hacer una mejor clasificación. Bajo este escenario, la varianza es baja, pero el sesgo es alto debido también al desbalanceo fuerte que hemos observado sobre la clase 2.

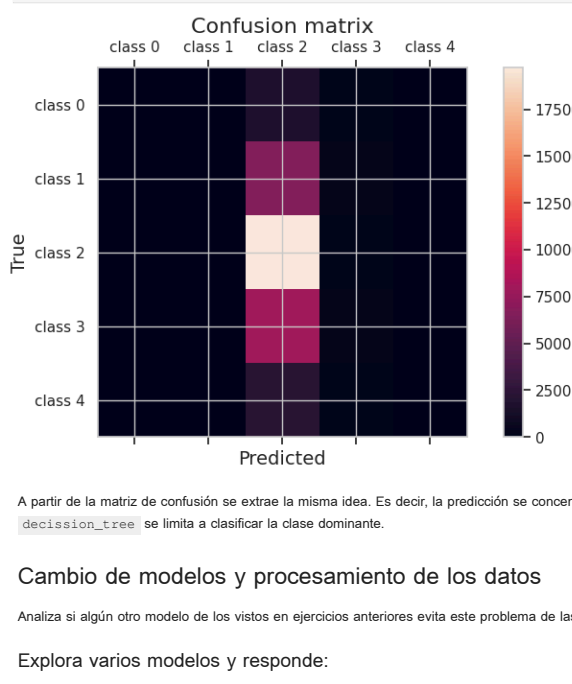
De hecho, al analizar las distribuciones de las predicciones, se identifica que la causa de que el `get_accuracy` sea bajo se debe a que el modelo tan solo asigna 2 etiquetas de clases de las 5 posibles. Concretamente, el 97% de las predicciones corresponden a la clase 2 y el 3% a la clase 3, coincidiendo con las clases con mayor frecuencia en el conjunto de datos de entrenamiento. Lo que hace que las clases 1, 4, 5 no sean predichas y contendiendo un error del 100% a la hora de predecir dichas clases.

Es decir, al ser un problema no balanceado, la predicción está claramente sesgada hacia una de las clases.

Esto se puede ver también analizando la **matriz de confusión**

In [11]:

```python
import numpy as np
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt

target_names = ['negative', 'somewhat negative', 'neutral', 'somewhat positive', 'positive']

def plot_confusion_matrix(pipe):
    ticks = np.unique(y_test)
    labels = ["class " + str(tick) for tick in ticks]
    y_pred = pipe.predict(X_test)
    cm = confusion_matrix(y_test, y_pred)
    fig = plt.figure(figsize=(10, 5))
    ax = fig.add_subplot(111)
    cax = ax.matshow(cm)
    plt.title('Confusion matrix')
    fig.colorbar(cax)
    ax.set_xticks(ticks)
    ax.set_yticks(ticks)
    ax.set_xticklabels(labels)
    ax.set_yticklabels(labels)
    plt.xlabel('Predicted')
    plt.ylabel('True')
    plt.show()

plot_confusion_matrix(pipe)
```



A partir de la matriz de confusión se extrae la misma idea. Es decir, la predicción se concentra de manera exclusiva sobre la clase 2, perdiendo la capacidad de discriminar las distintas clases. Por lo tanto, el clasificador `decision_tree` se limita a clasificar la clase dominante.

Cambio de modelos y procesamiento de los datos

Analiza si algún otro modelo de los vistos en ejercicios anteriores evita este problema de las clases desbalanceadas.

Explora varios modelos y responde:

- ¿Con qué modelo se obtienen mejores resultados?
- ¿Hay algún parámetro de los modelos que sea particularmente efectivo en evitar el desbalanceo?

Nota: incluye tantas celdas como precises para enseñar el código que has usado para responder a estas preguntas.

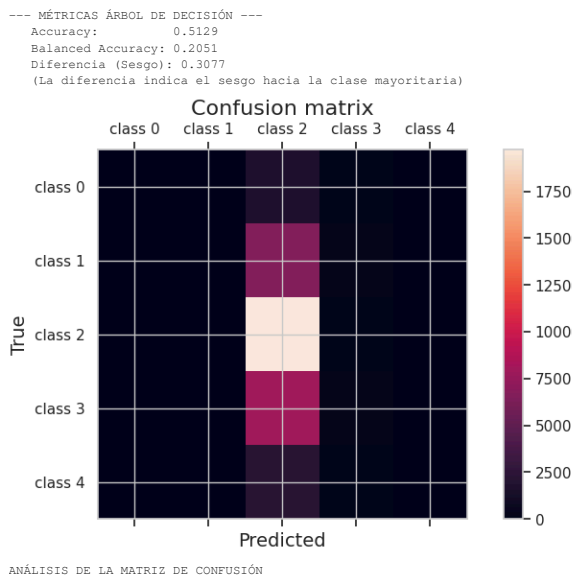
PARTE 1: MODELO BASE Y DESBALANCEO DE CLASES

En esta sección implementaremos un modelo que sirva de base sin ajustes adicionales para manejar el desbalanceo de clases. El objetivo es **demostrar explícitamente cómo falla un modelo básico** cuando se enfrenta a datos desbalanceados, sirviendo como punto de comparación para las soluciones que implementaremos posteriormente.

1. MODELO BASE - ÁRBOL DE DECISIÓN:

```python
# CountVectorizer (será igual para todos los modelos de esta parte 1)
baseline_vectorizer = CountVectorizer(max_features=1000)

# Creamos un modelo simplista con alto sesgo (ej. Árbol con 2 nodos)
baseline_classifier = DecisionTreeClassifier(criterion='entropy', max_depth=2, random_state=0)
baseline_pipeline = Pipeline([
    ('vectorizer', baseline_vectorizer),
    ('classifier', baseline_classifier)
])

print("MODELO: Árbol de Decisión sin ajustes para desbalanceo (Base)")
print("Pipeline: CountVectorizer(max_features=1000) -> DecisionTreeClassifier(max_depth=2)")

# Entrenar el modelo
baseline_pipeline.fit(X_train, y_train)

# Predicciones
y_pred_baseline = baseline_pipeline.predict(X_test)

# Métricas
baseline_score = metrics(y_test, y_pred_baseline, "ÁRBOL DE DECISIÓN")

# Confusion Matrix
plot_confusion_matrix(baseline_pipeline)

# Análisis de la matriz de confusión
report_prediction_distribution(y_test, y_pred_baseline, target_names)

# Resumen
print(classification_report(y_test, y_pred_baseline, target_names=target_names, digits=4))
```

```
MODELO: Árbol de Decisión sin ajustes para desbalanceo (Base)
Pipeline: CountVectorizer(max_features=1000) -> DecisionTreeClassifier(max_depth=2)

--- MÉTRICAS ÁRBOL DE DECISIÓN ---
Accuracy:          0.5129
Balanced Accuracy: 0.2051
Diferencia (Sesgo): 0.3077
(La diferencia indica el sesgo hacia la clase mayoritaria)
```



```
ANÁLISIS DE LA MATRIZ DE CONFUSIÓN

Distribución de predicciones del modelo:
  Clase 0 (negative): 0 predicciones (0.0%)
  Clase 1 (somewhat negative): 0 predicciones (0.0%)
  Clase 2 (neutral): 38020 predicciones (97.5%)
  Clase 3 (somewhat positive): 995 predicciones (2.5%)
  Clase 4 (positive): 0 predicciones (0.0%)

Distribución real en el conjunto de test:
  Clase 0 (negative): 1768 ejemplos (4.5%)
  Clase 1 (somewhat negative): 6818 ejemplos (17.5%)
  Clase 2 (neutral): 19896 ejemplos (51.0%)
  Clase 3 (somewhat positive): 8232 ejemplos (21.1%)
  Clase 4 (positive): 2301 ejemplos (5.9%)
                   precision    recall  f1-score   support

         negative     0.0000    0.0000    0.0000      1768
somewhat negative     0.0000    0.0000    0.0000      6818
          neutral     0.5149    0.9817    0.6813     19896
somewhat positive     0.2828    0.0340    0.0607      8232
         positive     0.0000    0.0000    0.0000      2301

         accuracy                         0.5129     39015
        macro avg     0.1603    0.2051    0.1484     39015
     weighted avg     0.3243    0.5129    0.3573     39015

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

Análisis de Resultados - Modelo Base

Los resultados validan la hipótesis de que el desbalanceo estructural del dataset invalida la métrica de `Accuracy` y genera un modelo sin valor predictivo real.

1. Métricas:
   - Accuracy (0.51) -> El modelo alcanza un 51% de exactitud, pero esto es engañoso. Dado que la clase Neutral (2) compone el ~51% del dataset, el modelo maximiza su métrica con una estrategia trivial: predecir siempre la clase mayoritaria ignorando las demás. No está aprendiendo semántica, solo explotando la distribución a priori. Esto equivale a un clasificador que está prediciendo la moda continuamente, cualquier modelo que no supere esta métrica no está aprendiendo nada realmente.
   - Balanced Accuracy (0.20) -> Al ponderar equitativamente las clases, la precisión colapsa al 20%. Esto equivale matemáticamente a tirar un dado de 5 caras ($\frac{1}{5} = 0.20$). La capacidad discriminatoria del modelo es nula.
2. La visualización de la matriz de confusión confirma el comportamiento "perezoso" del clasificador. El modelo concentra el 97.5% de sus predicciones en la clase central, ignorando que esta solo representa la mitad de los datos reales. Además se desvela un importante riesgo de coste, ya que el fallo crítico para un contexto financiero es la incapacidad de detectar opiniones polarizadas (Clases 0 y 4). El modelo es "ciego" ante los sentimientos muy positivos o muy negativos, perdiendo la totalidad de la señal.
3. Conclusión: El modelo base demuestra que maximizar el Accuracy en este dataset es una estrategia inviable. Por ello, es obligatorio intervenir en el entrenamiento; debemos implementar `class_weight="balanced"` para penalizar el error en clases minoritarias y forzar al modelo a aprender patrones lingüísticos en lugar de frecuencias estadísticas.

Otros modelos vistos en clase sin realizar ajustes de desbalanceo

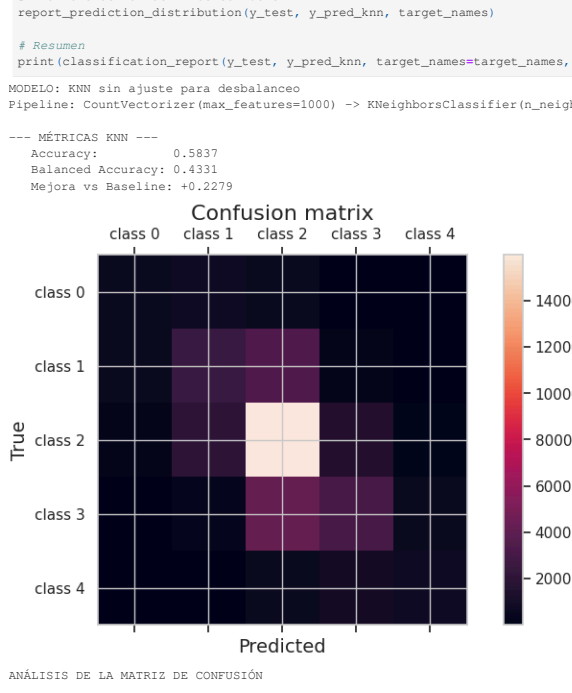
Alternativamente al árbol de decisión simple, estudiamos los clasificadores de vecinos próximos y redes neuronales con una capa oculta para comprobar si estos modelos a diferencia del árbol de decisión, son capaces de mitigar el problema de desbalanceo. O conocer quién actúa mejor bajo esta problemática.

En primer lugar, analizamos `KNeighborsClassifier` y posteriormente, `MLPClassifier`.

2. VECINOS PRÓXIMOS (KNN)

```python
# KNN sin ajustes de desbalanceo (k=5)
knn_vectorizer = CountVectorizer(max_features=1000)

knn_classifier = KNeighborsClassifier(
    n_neighbors=5,
    weights="distance",
    p=2
    )

knn_pipeline = Pipeline([
    ('vectorizer', knn_vectorizer),
    ('classifier', knn_classifier)
    ])

print("\nMODELO: KNN sin ajuste para desbalanceo")
print("Pipeline: CountVectorizer(max_features=1000) -> KNeighborsClassifier(n_neighbors=5)")

# Entrenar
knn_pipeline.fit(X_train, y_train)

# Predicciones
y_pred_knn = knn_pipeline.predict(X_test)

# Métricas
knn_score = metrics(y_test, y_pred_knn, "KNN", benchmark_score=baseline_score)

# Confusion Matrix
plot_confusion_matrix(knn_pipeline)

# Análisis de la matriz de confusión
report_prediction_distribution(y_test, y_pred_knn, target_names)

# Resumen
print(classification_report(y_test, y_pred_knn, target_names=target_names, digits=4))
```

```
MODELO: KNN sin ajuste para desbalanceo
Pipeline: CountVectorizer(max_features=1000) -> KNeighborsClassifier(n_neighbors=5)

--- MÉTRICAS KNN ---
Accuracy:          0.5837
Balanced Accuracy: 0.4331
Mejora vs Baseline: +0.2279
```



```
ANÁLISIS DE LA MATRIZ DE CONFUSIÓN

Distribución de predicciones del modelo:
  Clase 0 (negative): 1426 predicciones (3.7%)
  Clase 1 (somewhat negative): 5738 predicciones (14.7%)
  Clase 2 (neutral): 24698 predicciones (63.3%)
  Clase 3 (somewhat positive): 5722 predicciones (14.7%)
  Clase 4 (positive): 1409 predicciones (3.6%)

Distribución real en el conjunto de test:
  Clase 0 (negative): 1768 ejemplos (4.5%)
  Clase 1 (somewhat negative): 6818 ejemplos (17.5%)
  Clase 2 (neutral): 19896 ejemplos (51.0%)
  Clase 3 (somewhat positive): 8232 ejemplos (21.1%)
  Clase 4 (positive): 2301 ejemplos (5.9%)
                   precision    recall  f1-score   support

         negative     0.3923    0.3235    0.3546      1768
somewhat negative     0.4652    0.3755    0.4024      6818
          neutral     0.6470    0.8024    0.7165     19896
somewhat positive     0.5248    0.3685    0.4204      8232
         positive     0.4061    0.3038    0.3768      2301

         accuracy                         0.5837     39015
        macro avg     0.5003    0.4331    0.4561     39015
     weighted avg     0.5666    0.5837    0.5648     39015
```

Al evaluar KNN introducimos una regla de decisión diferente basada en la medición de la proximidad entre observaciones. Los resultados son más favorables en comparación con el árbol de decisión alcanzando un `Accuracy` y `Balanced Accuracy` superiores, 58,37% y 43,31%, respectivamente.

El modelo asume que textos con palabras similares (coordenadas cercanas) comparten el mismo sentimiento. Al mirar a los 5 vecinos más cercanos ($k=5$), el modelo logra capturar patrones semánticos reales. Ya no adivina; ahora compara. Por ese vemos cierta capacidad de discriminación en las clases intermedias y extremas que antes era nula. A pesar de la mejora, el modelo sigue mostrando una tendencia a predecir la clase mayoritaria (Clase 2). Cuando la clase 2 representa el 51% de los datos, es muy probable que los vecinos cercanos a un nuevo texto sean neutrales por pura densidad de población, ya que ocupan gran parte del espacio. Las clases extremas (0 y 4) son tan poco frecuentes y dispersas, por tanto la densidad de la minoritaria impide que las capacidades del modelo de predecir las clases minoritarias.

Además, el modelo `KNN` usa la distancia euclídea por defecto, y en textos (vectores de 1000 dimensiones llenos de ceros), la distancia Euclídea pierde sentido (todos los puntos están lejos).

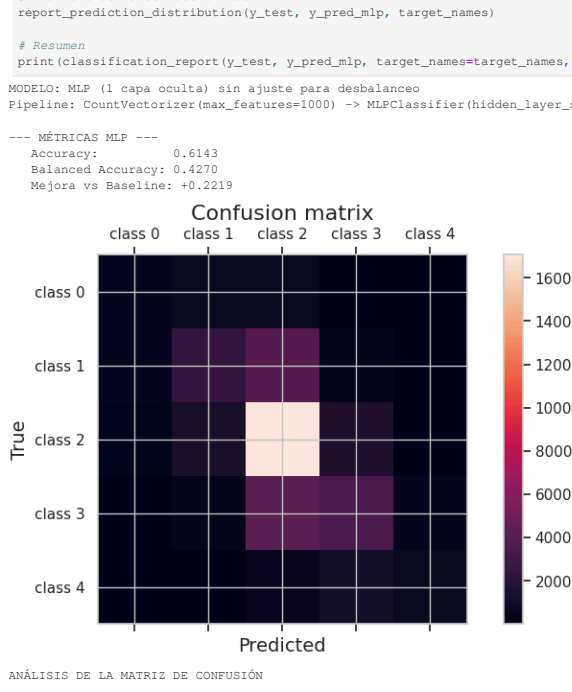
Aunque KNN mejora al árbol, no es ideal para texto debido a la alta dimensionalidad. Es preferible usar otras distancias como la del Coseno en lugar de Euclídea, o incluso otros modelos lineales como SVM/LogReg que implementaremos más adelante y manejan mejor espacios dispersos.

No obstante, los resultados de las métricas continúan siendo valores bajos y con poca sensibilidad hacia las clases minoritarias, algo que debemos priorizar enormemente en Finanzas.

3. REDES NEURONALES (MLP)

```python
# Modelo MLP con una sola capa de 50 neuronas sin ajustes de desbalanceo
mlp_vectorizer = CountVectorizer(max_features=1000)

mlp_classifier = MLPClassifier(
    hidden_layer_sizes=(50,),
    max_iter=300,
    early_stopping=True,
    random_state=0
)

mlp_pipeline = Pipeline([
    ('vectorizer', mlp_vectorizer),
    ('classifier', mlp_classifier)
])

print("\nMODELO: MLP (1 capa oculta) sin ajuste para desbalanceo")
print("Pipeline: CountVectorizer(max_features=1000) -> MLPClassifier(hidden_layer_sizes=(50,))")

# Entrenar
mlp_pipeline.fit(X_train, y_train)

# Predicciones
y_pred_mlp = mlp_pipeline.predict(X_test)

# Métricas
mlp_score = metrics(y_test, y_pred_mlp, "MLP", benchmark_score=baseline_score)

# Confusion Matrix
plot_confusion_matrix(mlp_pipeline)

# Análisis de la confusion matrix
report_prediction_distribution(y_test, y_pred_mlp, target_names)

# Resumen
print(classification_report(y_test, y_pred_mlp, target_names=target_names, digits=4))
```

```
MODELO: MLP (1 capa oculta) sin ajuste para desbalanceo
Pipeline: CountVectorizer(max_features=1000) -> MLPClassifier(hidden_layer_sizes=(50,))

--- MÉTRICAS MLP ---
Accuracy:          0.6143
Balanced Accuracy: 0.4270
Mejora vs Baseline: +0.2219
```



```
ANÁLISIS DE LA MATRIZ DE CONFUSIÓN

Distribución de predicciones del modelo:
  Clase 0 (negative): 896 predicciones (2.3%)
  Clase 1 (somewhat negative): 4517 predicciones (11.6%)
  Clase 2 (neutral): 26178 predicciones (67.1%)
  Clase 3 (somewhat positive): 6373 predicciones (16.3%)
  Clase 4 (positive): 1055 predicciones (2.7%)

Distribución real en el conjunto de test:
  Clase 0 (negative): 1768 ejemplos (4.5%)
  Clase 1 (somewhat negative): 6818 ejemplos (17.5%)
  Clase 2 (neutral): 19896 ejemplos (51.0%)
  Clase 3 (somewhat positive): 8232 ejemplos (21.1%)
  Clase 4 (positive): 2301 ejemplos (5.9%)
                   precision    recall  f1-score   support

         negative     0.4721    0.2393    0.3176      1768
somewhat negative     0.5254    0.3479    0.4185      6818
          neutral     0.6522    0.8581    0.7412     19896
somewhat positive     0.5472    0.4236    0.4775      8232
         positive     0.5805    0.2664    0.3652      2301

         accuracy                         0.6143     39015
        macro avg     0.5555    0.4270    0.4640     39015
     weighted avg     0.5955    0.6143    0.5878     39015
```

El Perceptrón Multicapa (MLP) presenta una mejora en métricas globales pero replica el fallo estructural que teníamos en vecinos próximos.

1. A pesar de su capacidad para modelar no-linealidades complejas, el MLP cae en la misma trampa estadística que el KNN.
   - Accuracy (61.43%): Ligeramente superior al KNN, pero engañoso.
   - Balanced Accuracy (42.70%): El modelo no ha aprendido a discriminar mejor las clases difíciles, simplemente ha optimizado mejor la clase fácil.
   - Gradiente: Durante el entrenamiento (Backpropagation), la función de pérdida es dominada por la clase mayoritaria. Como el 51% de los errores provienen de la clase "Neutral", el gradiente empuja los pesos del modelo para perfeccionar esa zona, sacrificando los extremos. El modelo aprende que "apostar a Neutral" es la ruta más rápida para minimizar el error global.
2. También en los resultados observamos un fenómeno interesante; la Precision en positivos es decente (58%), pero el Recall es desastroso (26%), en otras palabras, cuando el modelo se atreve a decir "Positivo", suele acertar. El problema es que rara vez se atreve. Es un modelo extremadamente conservador, diseñado para no fallar en el grueso de los datos e ignorando el "coste de oportunidad" de perder la señal en las colas.

Conclusión: La aplicación de modelos más complejos (KNN, MLP) sobre los datos brutos ha fallado. Hemos verificado que el problema no es de "capacidad del modelo", el algoritmo está haciendo exactamente lo que le pedimos: minimizar el error total. Pero en un dataset desbalanceado, el error total es una métrica equivocada.

Dado que el problema no es falta de datos, sino cómo el modelo penaliza los errores, la solución matemática correcta no es necesariamente inventar datos (oversampling), sino **modificar la función de coste**. En la siguiente sección, abandonaremos la maximización del Accuracy e implementaremos modelos que utilicen **`class_weight="balanced"`** para ver la mejora. Esto introducirá un peso w_j inversamente proporcional a la frecuencia de la clase en la función de pérdida, forzando al gradiente a prestar atención a los sentimientos extremos.

PARTE 2: Modificación de la Función de Coste

En esta sección implementamos la solución al problema de desbalanceo identificado en el baseline. A diferencia de técnicas de oversampling, que **crean nuevos datos** e introducen ruido, **modificaremos la función de coste** mediante el parámetro `class_weight="balanced"`.

¿Por qué no usamos Oversampling?

Las técnicas de oversampling **inventan datos sintéticos**. Esto puede:

- Introducir **ruido artificial** en el dataset.
- Generar ejemplos que no representan la realidad del problema.
- Aumentar el tiempo de entrenamiento al incrementar el tamaño del dataset.
- Crear dependencias entre ejemplos que pueden llevar a overfitting.

Solución: Modificación de la Función de Coste con `class_weight="balanced"`

En lugar de modificar los datos, modificamos **cómo el algoritmo aprende** ajustando la función de coste para penalizar más los errores en clases minoritarias.

Para un problema de clasificación con K clases, la función de coste estándar es:

$$L(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{j=1}^{K} y_{ij}\log(h_\theta(x_i)_j)$$

donde:

- m es el número de ejemplos
- y_{ij} es 1 si el ejemplo i pertenece a la clase j, 0 en caso contrario
- $h_\theta(x_i)_j$ es la probabilidad predicha de que el ejemplo i pertenezca a la clase j

Cuando usamos `class_weight="balanced"`, la función de coste se modifica para incluir pesos w_j que son **inversamente proporcionales a la frecuencia de cada clase**:

$$L_{\text{weighted}}(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{j=1}^{K} w_j \cdot y_{ij}\log(h_\theta(x_i)_j)$$

donde los pesos se calculan como:

$$w_j = \frac{n_{\text{samples}}}{n_{\text{classes}} \times n_j}$$

siendo:

- n_{samples}: número total de ejemplos
- n_{classes}: número de clases
- n_j: número de ejemplos de la clase j

Interpretación

1. **Clases mayoritarias** (ej. clase 2, Neutral): Tienen un peso w_j **menor**, por lo que los errores en estas clases contribuyen menos a la función de coste total.
2. **Clases minoritarias** (ej. clases 0 y 4, Negative y Positive): Tienen un peso w_j **mayor**, por lo que los errores en estas clases contribuyen **más** a la función de coste.
3. **Efecto en el gradiente**: Durante el entrenamiento, el algoritmo de optimización (descenso de gradiente etc) calcula el gradiente de la función de coste. Al tener pesos mayores en clases minoritarias, el gradiente **corrige más agresivamente** los errores en estas clases, forzando al modelo a aprender a clasificarlas correctamente.

$$\text{Coste Total} \approx \sum (w_{\text{class}} \cdot \text{Error})$$

Esto implica que el peso w_{class} actúa como un multiplicador de la penalización: un fallo en una clase con peso 5 "duele" 5 veces más que un fallo en una clase con peso 1.

Nota: Es fundamental destacar que esta ponderación se trata de una "modificación" de la función de coste para corregir de alguna manera la asimetría o la frecuencia desigual entre clases, pero hemos optado por mantener la homogeneidad simétrica de la matriz de costes; es decir, mientras que los pesos w_j compensan el sesgo de frecuencia del dataset (evitando que el modelo se sesgue hacia la clase mayoritaria), la matriz de costes subyacente sigue siendo simétrica. En nuestra implementación, y de aquí en adelante, el coste de un falso positivo es equivalente al de un falso negativo. No obstante, este enfoque garantiza la recuperación de señal en las colas de la distribución (clases 0 y 4), asegurando que el descenso de gradiente no sacrifique la capacidad predictiva en eventos minoritarios pero críticos en favor de una ganancia marginal de accuracy global.

Ventajas de modificar la función de costes

- **No modifica los datos**: Trabajamos con datos reales, sin inventar ejemplos.
- **Eficiente computacionalmente**: No aumenta el tamaño del dataset.
- **Interpretable**: Los pesos reflejan directamente la importancia relativa de cada clase.
- **Aplicable a múltiples algoritmos**: Funciona con Regresión Logística, Random Forest, SVM, etc.

En nuestro dataset:

- Clase 2 (Neutral): ~51% de los datos → peso $w_2 \approx 0.39$ (bajo)
- Clase 0 (Negative): ~4% de los datos → peso $w_0 \approx 5.0$ (alto)
- Clase 4 (Positive): ~6% de los datos → peso $w_4 \approx 3.3$ (alto)

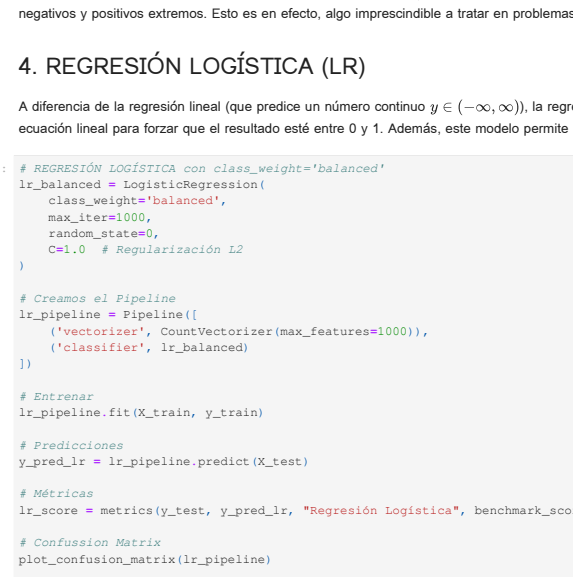
Esto significa que un error en la clase 0 tiene aproximadamente $5.0/0.39 \approx 12.8$ **veces más impacto** en la función de coste que un error en la clase 2, forzando al modelo a aprender a detectar correctamente los sentimientos negativos y positivos extremos. Esto es en efecto, algo imprescindible a tratar en problemas de gestión de riesgos financieros.

4. REGRESIÓN LOGÍSTICA (LR)

A diferencia de la regresión lineal (que predice un número continuo $y \in (-\infty, \infty)$), la regresión logística predice la probabilidad de pertenencia a una clase. Aplica una función de "aplastamiento" (Sigmoide o Softmax) a la ecuación lineal para forzar que el resultado esté entre 0 y 1. Además, este modelo permite añadir el hiperparámetro de `class_weight=balanced`.

```python
# REGRESIÓN LOGÍSTICA con class_weight="balanced"
lr_balanced = LogisticRegression(
    class_weight="balanced",
    max_iter=1000,
    random_state=0,
    C=1.0  # Regularización L2
)

# Creamos el Pipeline
lr_pipeline = Pipeline([
    ('vectorizer', CountVectorizer(max_features=1000)),
    ('classifier', lr_balanced)
])

# Entrenar
lr_pipeline.fit(X_train, y_train)

# Predicciones
y_pred_lr = lr_pipeline.predict(X_test)

# Métricas
lr_score = metrics(y_test, y_pred_lr, "Regresión Logística", benchmark_score=baseline_score)

# Confusion Matrix
plot_confusion_matrix(lr_pipeline)

# Análisis de la confusion matrix
report_prediction_distribution(y_test, y_pred_lr, target_names)

# Resumen
print(classification_report(y_test, y_pred_lr, target_names=target_names, digits=4))
```

```
--- MÉTRICAS REGRESIÓN LOGÍSTICA ---
Accuracy:          0.5094
Balanced Accuracy: 0.4617
Mejora vs Baseline: +0.2566
```



```
ANÁLISIS DE LA MATRIZ DE CONFUSIÓN

Distribución de predicciones del modelo:
  Clase 0 (negative): 4312 predicciones (11.1%)
  Clase 1 (somewhat negative): 5583 predicciones (14.3%)
  Clase 2 (neutral): 18909 predicciones (48.5%)
  Clase 3 (somewhat positive): 5632 predicciones (14.4%)
  Clase 4 (positive): 4622 predicciones (11.8%)

Distribución real en el conjunto de test:
  Clase 0 (negative): 1768 ejemplos (4.5%)
  Clase 1 (somewhat negative): 6818 ejemplos (17.5%)
  Clase 2 (neutral): 19896 ejemplos (51.0%)
  Clase 3 (somewhat positive): 8232 ejemplos (21.1%)
  Clase 4 (positive): 2301 ejemplos (5.9%)
                   precision    recall  f1-score   support

         negative     0.2051    0.4881    0.2839      1768
somewhat negative     0.3647    0.2964    0.3271      6818
          neutral     0.7028    0.6679    0.6849     19896
somewhat positive     0.4273    0.2923    0.3471      8232
         positive     0.2807    0.5637    0.3747      2301

         accuracy                         0.5094     39015
        macro avg     0.3961    0.4617    0.4036     39015
     weighted avg     0.5379    0.5094    0.5146     39015
```

La incorporación del parámetro `class_weight="balanced"` en la Regresión Logística ha generado un cambio importante en el comportamiento del clasificador, validando la modificación de la función de coste.

La evidencia de la mejora reside en el gran incremento del `Balanced Accuracy`, que ha ascendido desde un nivel prácticamente aleatorio (~20,5%) hasta el 46,17%, lo que supone una mejora neta de 25,66 puntos porcentuales respecto al modelo base.

Vale la pena destacar que `Accuracy` apenas ha variado (0.5094 vs 0.5129). Este fenómeno refleja un trade-off que se produce en la optimización; el algoritmo ha dejado de asegurar los aciertos "fáciles" y abundantes de la clase mayoritaria (neutral) para asumir el riesgo de intentar clasificar las clases minoritarias, penalizadas severamente por los pesos w_j.

Al examinar la distribución de las predicciones, confirmamos que el modelo ha recuperado la sensibilidad ante los extremos, superando la incapacidad que teníamos en el modelo base. Hemos pasado de ignorar completamente las clases minoritarias a asignar cerca de un 11% de las predicciones a la clase Muy Negativo (Clase 0) y un 11,8% a la clase Muy Positivo (Clase 4). De hecho, se observa que el modelo sobreestima ligeramente la presencia de estas categorías respecto a su frecuencia real (que ronda el 4-6%), esto es debido a que hemos introducido una penalización muy alta en la función de coste.

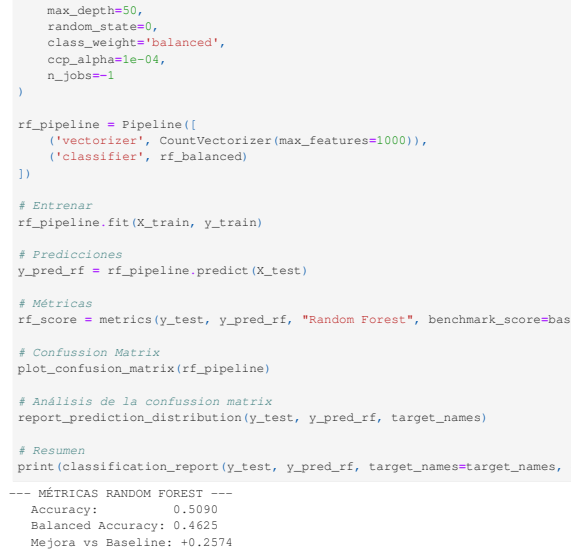
Desde una perspectiva de gestión de riesgos financieros, este comportamiento agresivo es, si no perfecto, al menos preferible al anterior.

5. RANDOM FOREST (RF)

A continuación, probamos con el clasificador `RandomForestClassifier` que también es compatible con el hiperparámetro `class_weight="balanced"` y observamos las diferencias en términos de accuracy, balance accuracy y la mejora con respecto al modelo base.

```python
# 2. RANDOM FOREST con class_weight="balanced"
rf_balanced = RandomForestClassifier(
    n_estimators=200,
    criterion="entropy",
    max_depth=50,
    random_state=0,
    class_weight="balanced",
    ccp_alpha=1e-04,
    n_jobs=-1
)

rf_pipeline = Pipeline([
    ('vectorizer', CountVectorizer(max_features=1000)),
    ('classifier', rf_balanced)
])

# Entrenar
rf_pipeline.fit(X_train, y_train)

# Predicciones
y_pred_rf = rf_pipeline.predict(X_test)

# Métricas
rf_score = metrics(y_test, y_pred_rf, "Random Forest", benchmark_score=baseline_score)

# Confusion Matrix
plot_confusion_matrix(rf_pipeline)

# Análisis de la confusion matrix
report_prediction_distribution(y_test, y_pred_rf, target_names)

# Resumen
print(classification_report(y_test, y_pred_rf, target_names=target_names, digits=4))
```

```
--- MÉTRICAS RANDOM FOREST ---
Accuracy:          0.5090
Balanced Accuracy: 0.4625
Mejora vs Baseline: +0.2574
```



```
ANÁLISIS DE LA MATRIZ DE CONFUSIÓN

Distribución de predicciones del modelo:
  Clase 0 (negative): 3038 predicciones (7.8%)
  Clase 1 (somewhat negative): 5240 predicciones (13.4%)
  Clase 2 (neutral): 20505 predicciones (52.6%)
  Clase 3 (somewhat positive): 5568 predicciones (14.3%)
  Clase 4 (positive): 4660 predicciones (11.9%)

Distribución real en el conjunto de test:
  Clase 0 (negative): 1768 ejemplos (4.5%)
  Clase 1 (somewhat negative): 6818 ejemplos (17.5%)
  Clase 2 (neutral): 19896 ejemplos (51.0%)
  Clase 3 (somewhat positive): 8232 ejemplos (21.1%)
  Clase 4 (positive): 2301 ejemplos (5.9%)
                   precision    recall  f1-score   support

         negative     0.2864    0.4921    0.3620      1768
somewhat negative     0.3791    0.2856    0.3286      6818
          neutral     0.6632    0.6837    0.6733     19896
somewhat positive     0.3659    0.2461    0.2936      8232
         positive     0.2964    0.6002    0.3968      2301

         accuracy                         0.5090     39015
        macro avg     0.3974    0.4625    0.4109     39015
     weighted avg     0.5113    0.5090    0.5025     39015
```

Random Forest es un clasificador que ajusta varios clasificadores de árboles de decisión en diversas submuestras del dataset y utiliza el promedio para mejorar la precisión predictiva y controlar el sobreajuste. Por lo tanto, es muy robusto dado que mezcla ruido y luego promedia, consiguiendo sesgo y varianza baja y además, tiene las ventajas de que es sencillo de entrenar y tiene pocos hiperparámetros.

Para nuestro estudio hemos decidido que los hiperparámetros del Random Forest sean:

- **n_estimators=200**: es el número de árboles de decisión de la submuestra.
- **criterion = "entropy"**: para comparabilidad con el árbol de decisión evaluado anteriormente.
- **max_depth=50**: determina la máxima profundidad permitida para cada árbol.
- **random_state=0**: por reproducibilidad
- **class_weight="balanced"**: para ajustar las ponderaciones de las clases de forma inversamente proporcional a sus frecuencias, con el objetivo de mitigar el desbalanceo.
- **ccp_alpha=1e-04**: sirve para penalizar árboles demasiado complejos.

La inclusión de `class_weight="balanced"` vuelve a mostrar que resuelve considerablemente el problema de desbalanceo, obteniendo resultados mejores en comparación con los obtenidos en la instancia inicial con un árbol de decisión muy simple e infraajustado. Sin embargo, no superan a los obtenidos con `LogisticRegression` por decimales.

Esto es, para valores elevados de `max_depth`, Random Forest alcanza un valor de Balance Accuracy igual a $46,25\%$ y una mejora respecto al Baseline de $25,74\%$. Lo que conduce a una mayor sensibilidad de las clases minoritarias como por ejemplo los sentimientos negativo y positivo. En cuanto a la distribución de las predicciones, este modelo vuelve a replicar los resultados obtenidos con Logistic Regression. Dando mayor peso a las clases negativa (8%), somewhat negative y somewhat positive (en torno al 14%) y positive ($\approx 12\%$).

No obstante, esta mejora con respecto al árbol de decisión inicial se consigue a costa de un incremento significativo en la complejidad del modelo, dado que Random Forest combina un gran número de árboles profundos y no lineales. A pesar del progreso, los resultados no muestran ventajas significativas con respecto a la Regresión Logística. Por ello, extraemos la idea de que modelos complejos como el Random Forest no son la clave para resolver el problema dado que un modelo más simple como el Logistic Regresión consigue resultados muy similares con el beneficio de tener menos coste computacional y mayor interpretabilidad. (Navaja de Ockham).

COMPARACIÓN: MODELO BASE VS MODELOS CON DESBALANCEO DE CLASES

Ahora comparamos explícitamente a los modelos con `class_weight="balanced"` detectan mejor las clases extremas (0: Negative y 4: Positive) en comparación con el Baseline.

Para evaluar la capacidad de los modelos en la detección de sentimientos extremos (que son minoritarios pero críticos), utilizamos la métrica de **Recall** (o Tasa de Verdaderos Positivos).

Dado que este es un problema multiclase, calculamos el recall para una clase específica c (ej. $c=0$) considerando esa clase como "Positiva" y todas las demás como "Negativas" (enfoque *One-vs-Rest*).

La fórmula matemática es:

$$\text{Recall}_c = \frac{TP_c}{TP_c + FN_c}$$

Donde:

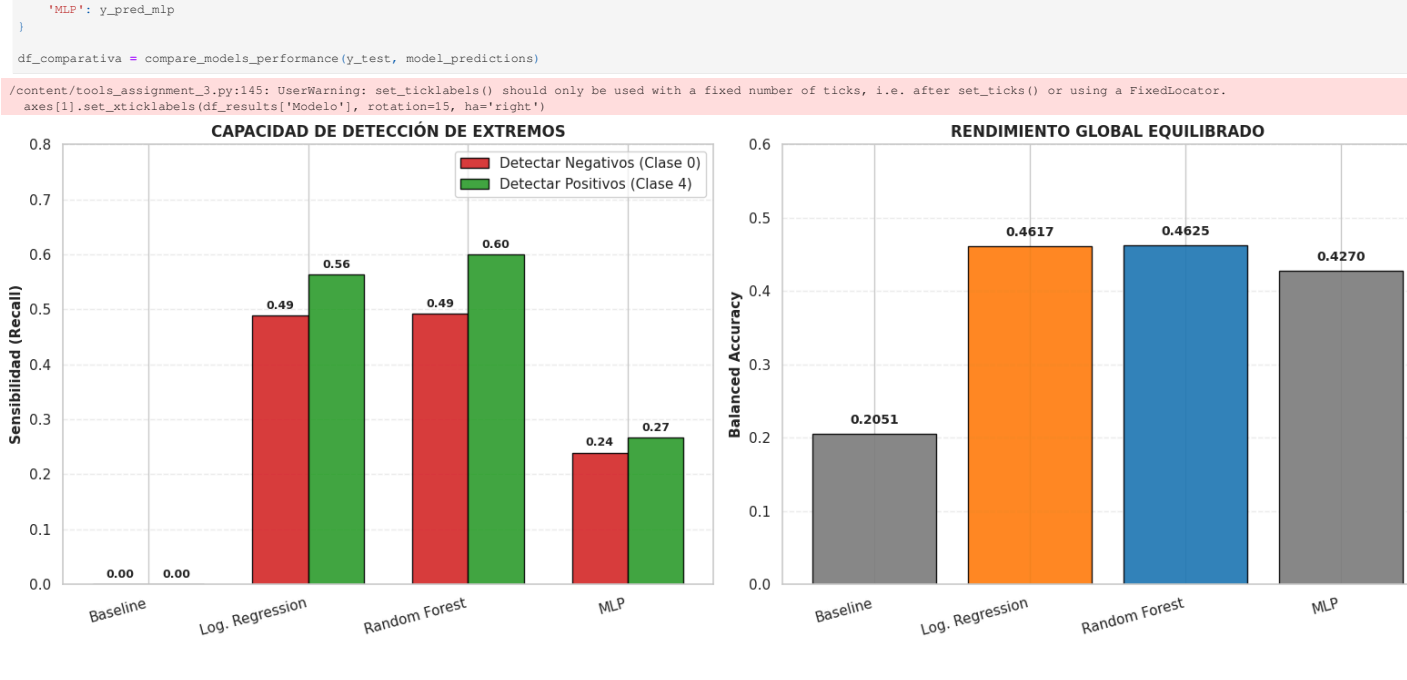
- TP_c (**True Positives**): Cantidad de muestras de la clase c clasificadas correctamente como c.
- FN_c (**False Negatives**): Cantidad de muestras que eran de la clase c, pero el modelo las clasificó incorrectamente como otra clase.

Interpretación:

- Un **Recall alto en Clase 0** significa que el modelo es muy bueno detectando quejas o críticas negativas (pocos falsos negativos).
- Un **Recall alto en Clase 4** indica que el modelo captura eficazmente los promotores o clientes muy satisfechos.

```python
# Calcular recall (sensibilidad) para cada clase en cada modelo
model_predictions = {
    "Baseline": y_pred_baseline,
    "Log. Regression": y_pred_lr,
    "Random Forest": y_pred_rf,
    "MLP": y_pred_mlp
}

df_comparative = compare_models_performance(y_test, model_predictions)
```

```
/content/tools_assignment_3.py:235: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  axes[1].set_xticklabels(df_results['Modelo'], rotation=15, ha='right')
```



PARTE 3: Procesamiento de los datos

Enunciado El primer paso que se usó en el *pipe* anterior es muy simple: sólo cuenta cuántas veces aparece cada término. Además, se ha limitado a un máximo de 1000 dimensiones. Prueba otras formas de procesar el texto y discute las diferencias.

Como ejemplo, en la celda siguiente se usa un método que considera la frecuencia de los términos (TfidfVectorizer). La documentación muestra que permite varias configuraciones (eliminar acentos, pasar a minúsculas, realizar otras transformaciones más complejas, quitar palabras comunes, etc.).

Hasta este punto, hemos establecido un baseline utilizando `CountVectorizer`, el cual convierte el texto en vectores basándose únicamente en la frecuencia absoluta de los términos. Si bien este método logra transformar datos no estructurados en numéricos, presenta limitaciones fundamentales: trata todas las palabras por igual, permitiendo que términos gramaticales sin carga semántica (stop-words como "the", "and", "is") dominen la representación vectorial debido a su alta frecuencia, eclipsando adjetivos cruciales para determinar el sentimiento.

Sin embargo, en análisis de sentimiento, la eliminación de stop-words es peligrosa. Muchas listas estándar de stop-words incluyen términos de negación como "not", "no" o "but". Si eliminamos estas palabras en un modelo simple (unigramas), una frase como "not good" perdería el "not", quedándose solo con "good", lo que invertiría totalmente la polaridad del sentimiento (de negativo a positivo) e inducirá al modelo a error.

Para mitigar esto y mejorar la relación señal/ruido del modelo, implementamos dos mejoras estratégicas en esta sección:

Transición a `TF-IDF` (Term Frequency-Inverse Document Frequency): Sustituimos el conteo simple por una ponderación estadística. TF-IDF penaliza los términos que aparecen en todas las reseñas (reduciendo el ruido) y aumenta el peso de las palabras distintivas que aparecen en pocas reseñas pero son muy informativas.

Incorporación de `N-gramas`: Ampliamos la vectorización para capturar no solo palabras aisladas (unigramas = 1 palabra), sino secuencias (bigramas = 2 palabras). Esto es vital para detectar estructuras de negación (ej. distinguir "good" de "not good" como 1 solo token) que un modelo simple ignoraría. Esto permite al modelo entender el contexto negativo.

Además, aumentaremos el número de features a 5000 (`max_features=5000`). Esto aumenta el vocabulario de 1000 a 5000 términos y permite al modelo capturar más matices del lenguaje

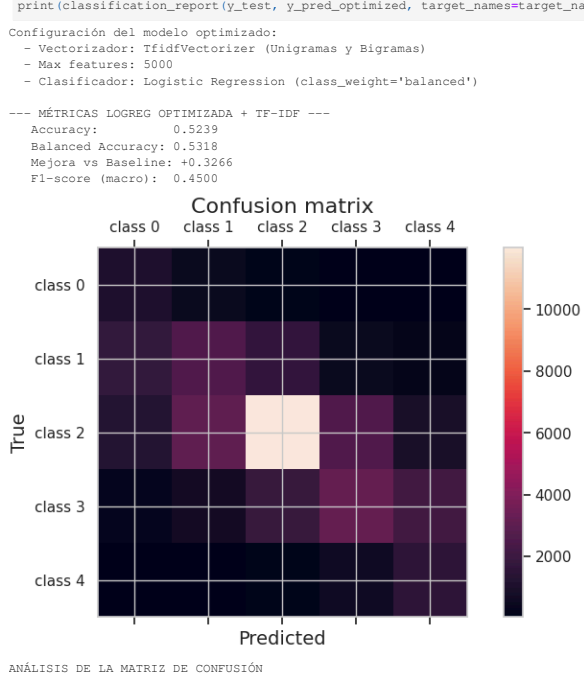
Objetivo: Evaluaremos si la combinación de una mejor representación de los datos (TF-IDF + Bigramas) junto con el ajuste de costes (`class_weight="balanced"`) permite superar al baseline, analizando el impacto en métricas que ponderan las clases minoritarias como el F-Score.

A continuación descartaremos algoritmos basados en distancia (KNN) por ineficiencia en alta dimensionalidad y optaremos por modelos lineales robustos (LogReg) que son los que mejores resultados nos han dado y permiten explicabilidad y N-gramas.

6. REGRESIÓN LOGÍSTICA (OPTIMIZADA)

```python
# TfidfVectorizer + N-GRAMAS (UNIGRAMAS, Y BIGRAMAS)
tfidf_vectorizer_optimized = TfidfVectorizer(
    max_features=5000,
    ngram_range=(1, 2),
    stop_words="english",
    lowercase=True,
    min_df=2
)

# Regresión logística con class_weight="balanced"
lr_optimized = LogisticRegression(
    class_weight="balanced",
    max_iter=1000,
    random_state=42,
    C=1.0
)

# Creamos el Pipeline
optimized_pipeline = Pipeline([
    ('vectorizer', tfidf_vectorizer_optimized),
    ('classifier', lr_optimized)
])

print("Configuración del modelo optimizado:")
print(" - Vectorizador: TfidfVectorizer (Unigramas y Bigramas)")
print(" - Max Features: 5000")
print(" - Clasificador: Logistic Regression (class_weight='balanced')")

# Entrenar
optimized_pipeline.fit(X_train, y_train)

# Predicciones
y_pred_optimized = optimized_pipeline.predict(X_test)

# Métricas Numéricas (Incorporamos F1_SCORE)
lr_tfidf_score = metrics(y_test, y_pred_optimized, "LogReg Optimizada + TF-IDF", benchmark_score=baseline_score)
f1_mac = f1_score(y_test, y_pred_optimized, average='macro')
print(f"   F1-Score (macro): {f1_mac:.4f}")

# Confusion Matrix
plot_confusion_matrix(optimized_pipeline)

# Análisis de la Confusion Matrix
report_prediction_distribution(y_test, y_pred_optimized, target_names)

# Resumen
print(classification_report(y_test, y_pred_optimized, target_names=target_names, digits=4))
```

```
Configuración del modelo optimizado:
 - Vectorizador: TfidfVectorizer (Unigramas y Bigramas)
 - Max Features: 5000
 - Clasificador: Logistic Regression (class_weight='balanced')

--- MÉTRICAS LOGREG OPTIMIZADA + TF-IDF ---
Accuracy:          0.5239
Balanced Accuracy: 0.5318
Mejora vs Baseline: +0.3266
   F1-Score (macro): 0.4500
```



```
ANÁLISIS DE LA MATRIZ DE CONFUSIÓN

Distribución de predicciones del modelo:
  Clase 0 (negative): 4570 predicciones (11.7%)
  Clase 1 (somewhat negative): 6785 predicciones (17.4%)
  Clase 2 (neutral): 15810 predicciones (40.5%)
  Clase 3 (somewhat positive): 7012 predicciones (18.0%)
  Clase 4 (positive): 4838 predicciones (12.4%)

Distribución real en el conjunto de test:
  Clase 0 (negative): 1768 ejemplos (4.5%)
  Clase 1 (somewhat negative): 6818 ejemplos (17.5%)
  Clase 2 (neutral): 19896 ejemplos (51.0%)
  Clase 3 (somewhat positive): 8232 ejemplos (21.1%)
  Clase 4 (positive): 2301 ejemplos (5.9%)
                   precision    recall  f1-score   support

         negative     0.2414    0.6239    0.3481      1768
somewhat negative     0.3825    0.3805    0.3815      6818
          neutral     0.7286    0.5820    0.6719     19896
somewhat positive     0.4403    0.3922    0.4236      8232
         positive     0.3135    0.6593    0.4249      2301

         accuracy                         0.5239     39015
        macro avg     0.4213    0.5318    0.4500     39015
     weighted avg     0.5803    0.5239    0.5395     39015
```

En esta sección, evaluamos la contribución estratégica del ajuste de costes (`class_weight="balanced"`) con una vectorización semántica superior (`TfidfVectorizer + N-gramas`). Los resultados confirman un salto cualitativo respecto al baseline:

El modelo alcanza un Accuracy del 52.39% y, más importante aún, un Balanced Accuracy del 53.18%. La mejora respecto al baseline es masiva (+0.3266), lo que valida la hipótesis de que el problema no era la capacidad del algoritmo, sino la calidad de los datos de entrada.

Al observar la distribución de predicciones, vemos los siguientes resultados:

Clase Neutral (2): En los datos reales representa el 51%, pero el modelo solo predice un 40.5%. El modelo ya no se "esconde" en la respuesta segura.

Clases Extremas (0 y 4): Aquí está la ganancia real. El modelo predice un 11.7% de clase 0 (vs 4.5% real) y un 12.4% de clase 4 (vs 5.9% real).

En otras palabras, el modelo está "sobre-reaccionando" intencionadamente a las clases extremas. En un contexto de detección de riesgos, esto es deseable: preferimos un modelo con alta Sensibilidad que detecte todas las críticas negativas (aunque genere algunos falsos positivos), a uno conservador que lo marque todo como neutral.

El desglose por clases confirma lo comentado anteriormente: hemos priorizado la Sensibilidad (Recall) sobre la Precisión pura. Al elevar la detección en las colas (clases 0 y 4) por encima del 60%, el modelo opera ahora como un sistema de alerta temprana, minimizando los "Falsos Negativos" (ignorar una crítica grave). Desde una perspectiva de gestión de riesgos, esta asimetría es óptima, ya que el coste de oportunidad de perder una señal polarizada es muy superior al coste operativo de filtrar cierto ruido (Falso Positivo).

Conclusión: La combinación de TF-IDF (que limpia ruido y detecta contexto con bigramas) junto con la penalización de costes ha resultado ser la estrategia más eficaz. Hemos pasado de un modelo ciego que predecía todo "Neutral", a uno capaz de discriminar y priorizar la señal emocional en los extremos de la distribución.

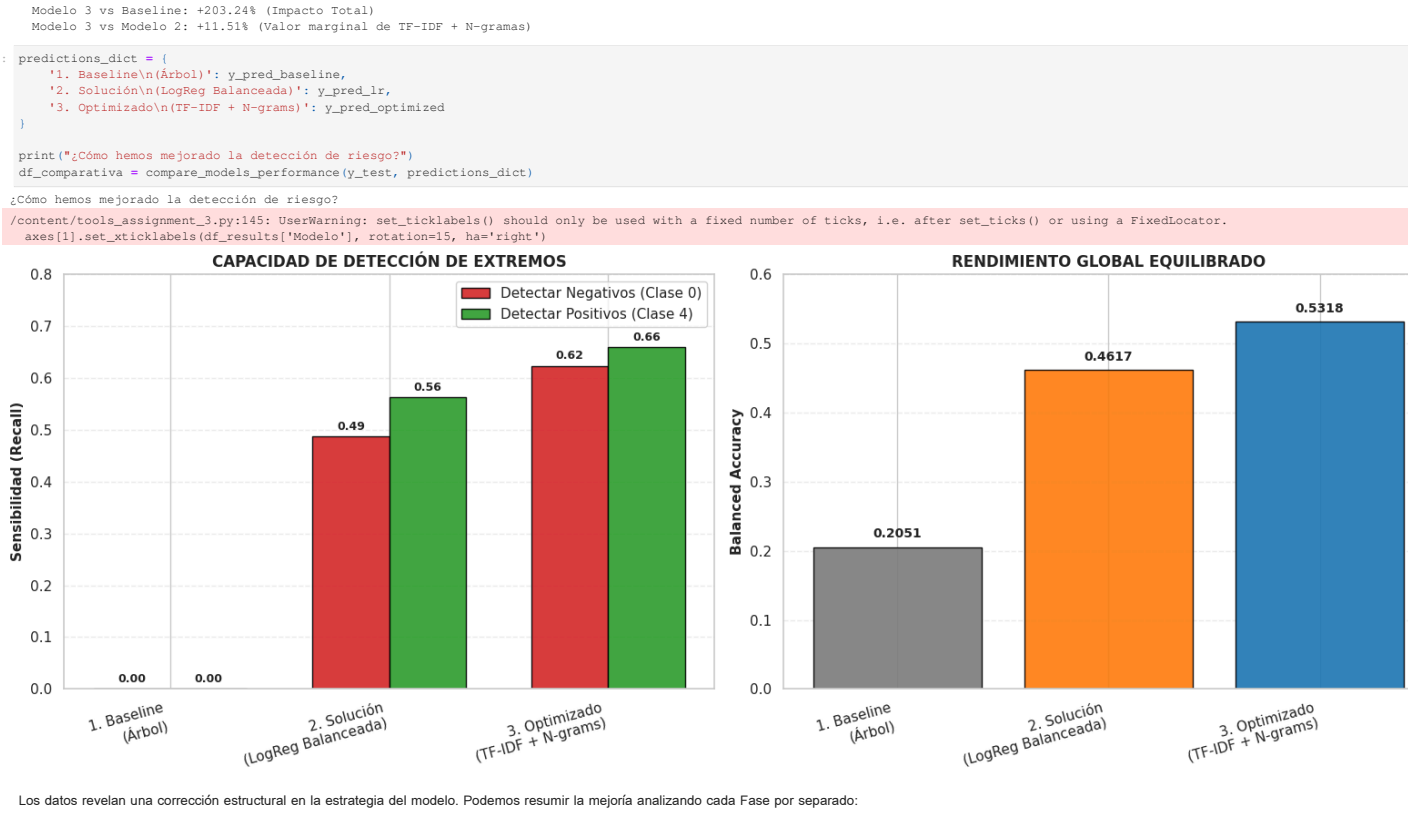
COMPARATIVA FINAL: ÁRBOL VS LOGREG VS LOGREG OPTIMIZADA

```python
# COMPARATIVA FINAL:

# 1. Modelo Base (Árbol) - Parte 1
b_acc_1 = balanced_accuracy_score(y_test, y_pred_baseline)
acc_1   = accuracy_score(y_test, y_pred_baseline)
f1_1    = f1_score(y_test, y_pred_baseline, average='macro')

# 2. Solución Intermedia (LogReg + CountVect) - Parte 2
b_acc_2 = balanced_accuracy_score(y_test, y_pred_lr)
acc_2   = accuracy_score(y_test, y_pred_lr)
f1_2    = f1_score(y_test, y_pred_lr, average='macro')

# 3. Optimizado (LogReg + TF-IDF + N-grams) - Parte 3
b_acc_3 = balanced_accuracy_score(y_test, y_pred_optimized)
acc_3   = accuracy_score(y_test, y_pred_optimized)
f1_3    = f1_score(y_test, y_pred_optimized, average='macro')

# TABLA
comparison_final = {
    'Modelo / Estado': [
        '1. Baseline (Árbol Simple)',
        '2. Intermedio (LogReg + CountVec)',
        '3. Optimizado (TF-IDF + Bigramas)'
    ],
    'F1-Score (Macro)':   [f1_1, f1_2, f1_3],
    'Balanced Accuracy':  [b_acc_1, b_acc_2, b_acc_3],
    'Accuracy Global':    [acc_1, acc_2, acc_3]
}

df_final = pd.DataFrame(comparison_final)

# Valor Añadido
df_final['Mejora F1 vs Baseline'] = df_final['F1-Score (Macro)'] - f1_1

print("\n TABLA 1: COMPARATIVA DE GENERACIÓN DE VALOR:")
print(df_final.to_string(index=False, float_format="%.4f"))

# Mejora Porcentual
imp_sol = ((f1_2 - f1_1) / f1_1) * 100
imp_opt = ((f1_3 - f1_1) / f1_1) * 100
imp_marg = ((f1_3 - f1_2) / f1_2) * 100

print("\n TABLA 2: ANÁLISIS (Mejora en F1-Macro):")
print(f"   Modelo 2 vs Baseline: {imp_sol:+.2f}% (Impacto de cambiar el algoritmo)")
print(f"   Modelo 3 vs Baseline: {imp_opt:+.2f}% (Impacto Total)")
print(f"   Modelo 3 vs Modelo 2: {imp_marg:+.2f}% (Valor marginal de TF-IDF + N-gramas)")
```

```
 TABLA 1: COMPARATIVA DE GENERACIÓN DE VALOR:
                  Modelo / Estado  F1-Score (Macro)  Balanced Accuracy  Accuracy Global  Mejora F1 vs Baseline
       1. Baseline (Árbol Simple)            0.1484             0.2051           0.5129                 0.0000
2. Intermedio (LogReg + CountVec)            0.4036             0.4617           0.5094                 0.2551
3. Optimizado (TF-IDF + Bigramas)            0.4500             0.5318           0.5239                 0.3016

 TABLA 2: ANÁLISIS (Mejora en F1-Macro):
   Modelo 2 vs Baseline: +171.94% (Impacto de cambiar el algoritmo)
   Modelo 3 vs Baseline: +203.19% (Impacto Total)
   Modelo 3 vs Modelo 2: +11.51% (Valor marginal de TF-IDF + N-gramas)
```

```python
predictions_dict = {
    '1. Baseline\n(Árbol)': y_pred_baseline,
    '2. Solución\n(LogReg Balanceada)': y_pred_lr,
    '3. Optimizado\n(TF-IDF + N-grams)': y_pred_optimized
}

print("¿Cómo hemos mejorado la detección de riesgo?")
df_comparative = compare_models_performance(y_test, predictions_dict)
```

```
¿Cómo hemos mejorado la detección de riesgo?
/content/tools_assignment_3.py:235: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  axes[1].set_xticklabels(df_results['Modelo'], rotation=15, ha='right')
```



Los datos revelan una corrección estructural en la estrategia del modelo. Podemos resumir la mejoría analizando cada Fase por separado:

1. Fase 1

El Baseline tiene un Accuracy Global alto (51.29%), pero un F1-Score muy bajo (0.1484) y una Balanced Accuracy de 0.20 equivalente al azar entre 5 clases ($\sim 1/5$).

Conclusión: El modelo inicial no "aprendía", simplemente apostaba todo a la clase mayoritaria (Neutral). Era un modelo "ciego" que maximizaba la métrica fácil ignorando el riesgo real. Lo descartamos por ser demasiado simplista.

2. Fase 2

El salto del +171.94% en F1-Score al cambiar a Regresión Logística Balanceada representa la importancia del valor añadido en esta fase.

Aunque el Accuracy Global bajó ligeramente (de 51.29% a 50.94%), esto fue un "coste de reestructuración" necesario: dejamos de ignorar las clases difíciles. Pasamos de un modelo inservible a uno funcional.

3. Fase 3 - Optimizado:

La introducción de TF-IDF y N-grams generó un retorno marginal del +11.51% sobre la solución anterior. El modelo optimizado no solo mejora el F1 (0.4500), sino que recupera y supera el Accuracy Global del Baseline (52.39% vs 51.29%).

Conclusión: Hemos logrado lo mejor de ambos mundos: un modelo que entiende el contexto ("not good") y que es robusto tanto en métricas globales como en las clases minoritarias. Por lo que este se postula como el modelo definitivo.

Realiza pruebas con varios clasificadores y configuraciones para el procesamiento de texto y contesta:

- De acuerdo con tus experimentos, ¿afecta más cambiar el clasificador o cómo se procesa el texto?
- ¿Has encontrado algún tipo de procesamiento que siempre mejore los resultados? ¿En qué hipótesis te has basado para probarlo?

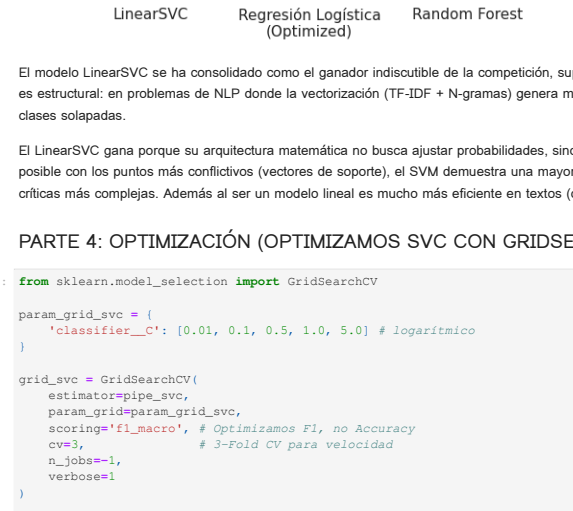
Hasta ahora hemos logrado una mejoría del modelo con $C=1$ (regularización), y un vocabulario de 5000 palabras, pero realizaremos una optimización robusta con la función `GridSearch` para ver qué hiperparámetros maximizan nuestro $F1$.

Nota: Usaremos cross-validation = 3 para aligerar el tiempo de cómputo
Nota: Compararemos la regresión logística optimizada que hemos obtenido, el SVC (Support Vector Machines) y un Random Forest con 100 árboles.

```python
from sklearn.svm import LinearSVC
from sklearn.ensemble import RandomForestClassifier

# 1. LinearSVC (Support Vector Machine Lineal)
pipe_svc = Pipeline([
    ('vectorizer', tfidf_vectorizer_optimized),
    ('classifier', LinearSVC(class_weight='balanced', random_state=42, dual='auto', max_iter=2000))
])
pipe_svc.fit(X_train, y_train)
y_pred_svc = pipe_svc.predict(X_test)

# Métricas SVC
f1_svc = f1_score(y_test, y_pred_svc, average='macro')
print(f"LinearSVC F1-Macro:     {f1_svc:.4f}")

# 2. Random Forest Classifier
pipe_rf = Pipeline([
    ('vectorizer', tfidf_vectorizer_optimized),
    ('classifier', RandomForestClassifier(
        n_estimators=50,
        max_depth=20,
        max_samples=0.3,
        class_weight='balanced',
        n_jobs=-1,
        random_state=42
    ))
])
pipe_rf.fit(X_train, y_train)
y_pred_rf = pipe_rf.predict(X_test)

# Métricas Random Forest
f1_rf = f1_score(y_test, y_pred_rf, average='macro')
print(f"Random Forest F1-Macro: {f1_rf:.4f}")
```

```
LinearSVC F1-Macro:     0.4805
Random Forest F1-Macro: 0.3427
```

```python
# Ranking
ranking_data = {
    'Modelo': [
        'Random Forest',
        'Regresión Logística\n(Optimized)',
        'LinearSVC'
    ],
    'F1-Macro': [
        f1_rf,
        f1_mac,
        f1_svc
    ],
    'Tecnología': [
        'Ensamble (Árboles/No Lineal)',
        'Lineal (Logits - Probabilidad)',
        'Lineal (Margen Máximo)'
    ]
}

df_ranking = pd.DataFrame(ranking_data).sort_values('F1-Macro', ascending=False)
ranking_final(df_ranking)
```



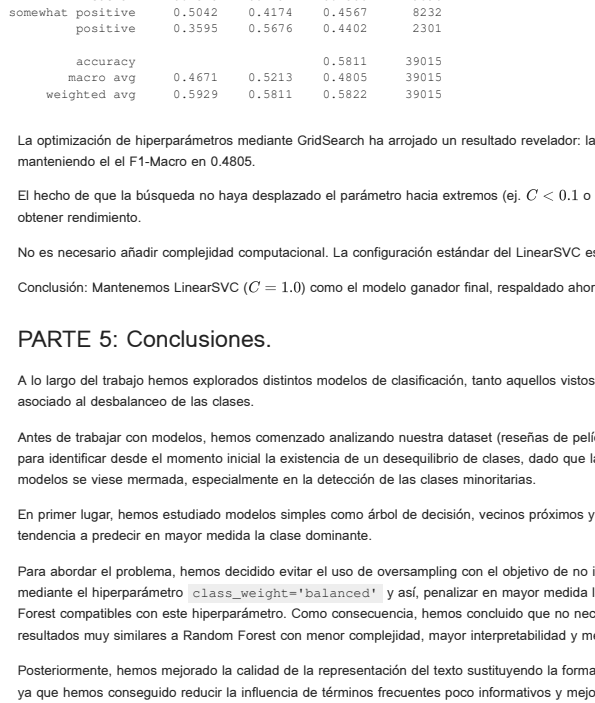
El modelo LinearSVC se ha consolidado como el ganador indiscutible de la competición, superando tanto a la Regresión Logística como al Random Forest con un F1-Macro superior (0.4805). Su victoria no es casualidad, sino que es estructural: en problemas de NLP donde la vectorización (TF-IDF + N-gramas) genera miles de dimensiones, los modelos probabilísticos como la Regresión Logística a veces sufren para definir fronteras de decisión nítidas entre clases solapadas.

El LinearSVC gana porque su arquitectura matemática no busca ajustar probabilidades, sino maximizar geométricamente el margen de separación entre las clases. Al trazar el hiperplano óptimo que mantiene la mayor distancia posible con los puntos más conflictivos (vectores de soporte), el SVM demuestra una mayor robustez y capacidad de generalización frente al ruido inherente del lenguaje natural, logrando discriminar mejor el sentimiento real en las críticas más complejas. Además al ser un modelo lineal es mucho más eficiente en textos (que suelen ser linealmente separables en alta dimensión) que modelos no lineales como Árboles o KNN.

PARTE 4: OPTIMIZACIÓN (OPTIMIZAMOS SVC CON GRIDSEARCH)

```python
from sklearn.model_selection import GridSearchCV

param_grid_svc = {
    'classifier__C': [0.01, 0.1, 0.5, 1.0, 5.0] # Regularización
}

grid_svc = GridSearchCV(
    estimator=pipe_svc,
    param_grid=param_grid_svc,
    scoring='f1_macro', # Optimizamos F1, no Accuracy
    cv=3,               # 3-Fold CV para velocidad
    n_jobs=-1,
    verbose=1
)

# 3. Entrenamiento (Optimización)
grid_svc.fit(X_train, y_train)

# 4. Resultados del Mejor Modelo Encontrado
best_svc_model = grid_svc.best_estimator_
y_pred_svc = best_svc_model.predict(X_test)

# Métricas Finales
f1_svc = f1_score(y_test, y_pred_svc, average='macro')
print(f"\nMejor Configuración (C): {grid_svc.best_params_['classifier__C']}")
print(f"LinearSVC F1-Macro (Optimizado): {f1_svc:.4f}")

# Confusion Matrix
plot_confusion_matrix(best_svc_model)

# Análisis confusion matrix
report_prediction_distribution(y_test, y_pred_svc, target_names)

# Resumen
print(classification_report(y_test, y_pred_svc, target_names=target_names, digits=4))
```

```
Fitting 3 folds for each of 5 candidates, totalling 15 fits

Mejor Configuración (C): 1.0
LinearSVC F1-Macro (Optimizado): 0.4805
```



```
ANÁLISIS DE LA MATRIZ DE CONFUSIÓN

Distribución de predicciones del modelo:
  Clase 0 (negative): 3260 predicciones (8.4%)
  Clase 1 (somewhat negative): 5461 predicciones (14.0%)
  Clase 2 (neutral): 19846 predicciones (50.9%)
  Clase 3 (somewhat positive): 6815 predicciones (17.5%)
  Clase 4 (positive): 3633 predicciones (9.3%)

Distribución real en el conjunto de test:
  Clase 0 (negative): 1768 ejemplos (4.5%)
  Clase 1 (somewhat negative): 6818 ejemplos (17.5%)
  Clase 2 (neutral): 19896 ejemplos (51.0%)
  Clase 3 (somewhat positive): 8232 ejemplos (21.1%)
  Clase 4 (positive): 2301 ejemplos (5.9%)
                   precision    recall  f1-score   support

         negative     0.2868    0.5288    0.3719      1768
somewhat negative     0.4534    0.3632    0.4033      6818
          neutral     0.7313    0.7297    0.7305     19896
somewhat positive     0.5042    0.4174    0.4567      8232
         positive     0.3595    0.5676    0.4402      2301

         accuracy                         0.5811     39015
        macro avg     0.4671    0.5213    0.4805     39015
     weighted avg     0.5929    0.5811    0.5822     39015
```

La optimización de hiperparámetros mediante GridSearch ha arrojado un resultado revelador: la configuración óptima de regularización es $C=1.0$, que es la misma que tienen los hiperparámetros estándar que teníamos antes y manteniendo el el F1-Macro en 0.4805.

El hecho de que la búsqueda no haya desplazado el parámetro hacia extremos (ej. $C<0.1$ o $C>10$) confirma que el modelo es estructuralmente estable. No dependemos de una calibración "artificial" o sobreajustada para obtener rendimiento.

Es no es necesario añadir complejidad computacional. La configuración estándar del LinearSVC es capaz de extraer el máximo valor de los datos vectorizados (TF-IDF), lo que simplifica su mantenimiento en producción.

Conclusión: Mantenemos LinearSVC ($C=1.0$) como el modelo ganador final, respaldado ahora no solo por su superioridad frente a la Regresión Logística, sino por la confirmación empírica de su estabilidad paramétrica.

PARTE 5: Conclusiones.

A lo largo del trabajo hemos explorado distintos modelos de clasificación, tanto aquellos vistos en clase como otros que nos resultaban desconocidos, con el fin de determinar cuál de ellos era válido para hacer frente al problema asociado al desbalanceo de las clases.

Antes de trabajar con modelos, hemos comenzado analizando nuestro dataset (reseñas de películas para analizar los sentimientos) y dibujando el histograma de la frecuencia de las clases. Esta herramienta gráfica, nos ha servido para identificar desde el momento inicial la existencia de un desequilibrio de clases, dado que la clase neutral tenía una frecuencia muy superior al resto. Este desbalance iba a ser el causante de que la capacidad predictiva de los modelos se viese mermada, especialmente en la detección de las clases minoritarias.

En primer lugar, hemos estudiado modelos simples como árbol de decisión, vecinos próximos y redes neuronales sin aplicar ningún ajuste para justificar la necesidad de tomar medidas dado que todos tienen como factor común la tendencia a predecir en mayor medida la clase dominante.

Para abordar el problema, hemos decidido evitar el uso de oversampling con el objetivo de no introducir datos sintéticos ni incrementar el coste computacional. Como alternativa, hemos optado por modificar la función de pérdida mediante el hiperparámetro `class_weight="balanced"` y así, penalizar en mayor medida los errores que se cometen sobre las clases minoritarias. Para ello, hemos analizado modelos como Regresión Logística y Random Forest compatibles con este hiperparámetro. Como consecuencia, hemos concluido que no necesariamente modelos más complejos garantizan mejor rendimiento. De hecho, hemos comprobado que la Regresión Logística alcanza resultados muy similares a Random Forest con menor complejidad, mayor interpretabilidad y menor coste computacional.

Posteriormente, hemos mejorado la calidad de la representación del texto sustituyendo la forma de vectorizar de `CountVectorizer` por `TfidfVectorizer` con n-gramas. Este cambio, nos ha proporcionado un salto de calidad, ya que hemos conseguido reducir la influencia de términos frecuentes poco informativos y mejorar la discriminación entre clases. En particular, la combinación de TF-IDF con `class_weight="balanced"` conduce a una distribución de predicciones más homogénea y a una mejor detección de reseñas con sentimientos negativos y positivos, evitando decisiones dominadas por la clase neutral.

Por último, LinearSVC destaca como el modelo con mejor rendimiento global (F1-Macro) debido a su capacidad por definir fronteras de decisión más robustas en espacios de alta dimensionalidad. Además, la optimización mediante `GridSearch` verifica la estabilidad del modelo al considerar (C=1.0) como valor óptimo de regularización.

Tabla resumen de los modelos:

Modelo	Class Weight	Vectorización	Balanced Accuracy	Comentarios
Decision Tree (baseline)	No	Count	Baja (~20%)	Fuerte sesgo hacia la clase neutral
KNN	No	Count	Media	Predicciones dominadas por C_2
MLP (1 capa)	No	Count	Media	Mayor capacidad, mismo problema
Logistic Regression	Sí	Count	Alta	Mejora clara en clases minoritarias
Random Forest	Sí	Count	Alta	Buen rendimiento, mayor complejidad
Logistic Regression	Sí	TF-IDF	Alta	Distribución más homogénea
LinearSVC	Sí	TF-IDF	-	Mejor rendimiento global (F1-Score)

Ideas claves extraídas:

- Ante problemas de desbalanceo de clases es necesario la intervenir para mejorar la calidad predictiva de los modelos.
- Estrategia aplicada para solucionar el desajuste: alterar la función de pérdida y el preprocesamiento de los datos.
- Modelos simples rinden bien, no es necesario aplicar modelos demasiado complejos para lograr mejorar el rendimiento del modelo.
- LinearSVC se selecciona como el modelo final por ofrecer el mejor equilibrio entre rendimiento, robustez y simplicidad computacional.